Thes1s

Agent Harness Engineering for Thes1s v3

Research Artifact - May 2026

May 2, 2026

Thes1s v3 Pipeline Architecture

Agent Harness Engineering for Thes1s v3

Topic: Agent / harness engineering patterns for LLM agent pipelines, with concrete recommendations for the Thes1s v3 backend. Audience: Kyle Hoff (non-programmer); engineers reading alongside. Date: May 2, 2026 Status: Research artifact — informs an in-progress design discussion. No code changes implied.

Executive Summary

This is a deep dive into how production LLM agent systems are built in 2026, written specifically to inform two open architectural questions in the Thes1s v3 pipeline: (1) how to re-enable web search inside an agent that must also emit strictly-typed JSON, and (2) how to surface live progress to the frontend during 5–10 minute multi-agent runs. Six research streams ran in parallel; the consolidated findings sit below.

The headline conclusions, in plain English:

1. Most "agents" in production aren't really agents. Anthropic's own taxonomy (workflows vs autonomous agents) puts the simple, predictable, code-driven pipelines on one side and the model-driven free-roaming agents on the other. Production teams overwhelmingly stay on the workflow side. Thes1s v3 is already a workflow (a coordinator dispatching specialists in fixed waves), and that's exactly where it should be.
2. The canonical pattern for "search then emit JSON" is a two-phase agent loop. Phase A: let the model freely call `web_search` with `tool_choice: 'auto'` until it finishes naturally. Phase B (only if Phase A didn't already emit structured output): make one more API call with the conversation history attached and `tool_choice` forced to the output tool. This is what Anthropic's own Cookbook recipes do, what LangGraph implements internally, and what the Anthropic multi-agent research blog post recommends. Almost every production agent that needs both tools and structured output uses some flavor of this pattern.
3. A new Anthropic feature could simplify our wrapper but does *not* solve the search-vs-output conflict. Anthropic shipped native structured outputs (`output_config.format` + `messages.parse()` with Zod) in late 2025. It's a strict upgrade over the "fake emit_output tool" trick for the no-search case — cleaner code, real type inference, decoder-level enforcement. But it has the same fundamental incompatibility with `web_search`. Switching to it does not unblock web search; only the two-phase loop does.
4. Forced `tool_choice` is also blocking extended thinking, not just web search. Anthropic's docs are explicit: extended thinking only works with `tool_choice: 'auto'` or `'none'`. The current v3 wrapper foregoes both web search and chain-of-thought reasoning by forcing the output tool. Re-architecting to a `tool_choice: 'auto'` agent loop unlocks both quality levers in one move.
5. For streaming progress, polling D1 is the right answer for the One Pager and the wrong answer for Pitch Deck and Full Story. Polling stays cheap and resilient through several thousand concurrent users, but a 10-minute spinner with no semantic feedback will feel broken to users no matter how reliable the backend is. The natural upgrade is Inngest Realtime (a built-in feature of the Inngest stack we already use), publishing wave/agent transitions and per-agent sub-progress, with the existing D1 polling pattern as a fallback when the websocket dies. The Worker can either subscribe-and-relay over SSE or mint a token and let the browser subscribe directly — both work, both stay inside the existing architecture.
6. Robustness comes from boring, layered defenses. Inngest's `NonRetriableError` for 4xx, `RetryAfterError` for 429s,

default exponential backoff for 5xx; turn-count and token-budget circuit breakers around the agent loop; reflect-and-retry up to 3 attempts on Zod failures; conditional UPDATEs in D1 to keep callbacks idempotent; `runId` as the global correlation key across Langfuse, Inngest, Worker logs, and D1 rows. None of this is novel; all of it is necessary.

7. Cost discipline matters more than model choice. Web search is the dominant cost driver in our pipelines (40–55% of total $/run). Capping searches at 3 per specialist, 8 for the One Pager, and 5 for debate agents preserves quality (research shows diminishing returns past ~5–7 searches per task) while shaving ~$1.20 off every Pitch Deck run. Reserving Opus 4.7 for the three agents where reasoning quality is the bottleneck (risk-analyst, valuation-specialist, quarterly-reader) and using Sonnet 4.6 elsewhere keeps the per-Pitch-Deck cost in the $2.40–$3.20 range, which industry benchmarks place comfortably inside the "sustainable" band for analytical agents.

The single most important recommendation: adopt Pattern 1 (auto loop → forced fallback) inside `agents-service/src/lib/anthropic-client.ts`. It's a localized wrapper-level change of roughly 80 lines of TypeScript. It does not touch the Inngest function structure, does not touch the Worker callback path, does not touch the agent prompts, does not touch the pipeline orchestration. It re-enables web search for all three stages, unlocks extended thinking, preserves prompt caching, and is the same shape Anthropic, LangChain, Vercel, Cognition, and every other production team has already converged on.

The streaming question is its own decision and can be made independently. The recommendation there is more nuanced and depends on whether you want to ship a "good enough" UX (keep polling, add a heartbeat) or a "delightful" UX (Inngest Realtime + event log + token-streamed final synthesis).

The "Recommendation for Thes1s" section at the end maps research findings to specific architectural questions for the brainstorm.

Agent Harness Patterns Survey (2026)

This section catalogs the patterns developers actually use in production multi-step LLM agent systems as of May 2026, distinguishing battle-tested approaches from research curiosities. Each pattern is described in plain English, with notes on when it works, when it fails, what it costs, and where it's deployed.

Anthropic's Core Distinction: Workflows vs. Agents

Anthropic's December 2024 "Building Effective Agents" post — still the most-cited reference document in 2026 — draws a sharp line between two categories [Source: https://www.anthropic.com/research/building-effective-agents]:

- Workflows are systems where LLMs and tools are orchestrated through *predefined code paths*. The developer wrote the flow chart; the LLM fills in the boxes.
- Agents are systems where the LLM *itself* dynamically directs its own processes and tool usage, deciding what to do next at each step.

Anthropic's blunt recommendation: "When building applications with LLMs, we recommend finding the simplest solution possible, and only increasing complexity when needed." Most production "agent" systems in 2026 are actually workflows — and that's by design, not by accident. Agents introduce non-determinism, higher cost, and harder debugging; workflows trade flexibility for predictability.

This framing has aged well. By 2026 the dominant production stacks (Cursor, Devin, Claude Code, the Anthropic Managed

Agents API, OpenAI's Responses API, LangGraph) all distinguish "workflow" from "agentic" loops as a first-class architectural choice.

Single-Shot Structured Output

What it is. One model call. The model is forced via tool_choice or response_format to emit a JSON object matching a schema. No tools, no loop, no iteration. Just: prompt in, structured object out.

When it shines. Classification, extraction, scoring, summarization of provided context, anything where the work is "transform this text into this shape." If the data the model needs is already in the prompt, single-shot is the cheapest, most reliable, most debuggable option.

When it hurts. When the model needs to fetch information, browse the web, reason through multiple steps, or self-correct. Single-shot has no recovery from a bad first guess.

Cost. 1 call. Tokens = input + output. Cheapest possible profile.

Production case studies. Used heavily under the hood at most production teams for sub-tasks within larger systems. The Thes1s v3 One Pager pipeline currently uses this pattern — `tool_choice: { type: 'tool', name: 'emit_output' }` forces a Zod-schema-shaped object on the first turn. Anthropic's docs explicitly recommend forced tool use for any structured-output endpoint [Source: https://docs.anthropic.com/en/docs/build-with-claude/tool-use/overview].

Agent Loop with Tool Use (Canonical Loop)

What it is. A while-loop. The model emits either a final answer or a tool call. If it's a tool call, the harness runs the tool, appends the result to the conversation, and calls the model again. Repeat until the model says it's done (or you hit a step limit).

When it shines. Open-ended tasks where the model needs to discover information, make decisions based on what it finds, and adapt. Coding agents, research agents, computer-use agents, customer-support agents.

When it hurts. When the path is known in advance (use a workflow). When tool latency is high and you can't afford 5–15 round-trips. When you need strict cost/time guarantees.

Cost. N calls, where N is how many tool turns the model takes. Each turn re-sends the growing conversation, so token cost grows roughly quadratically without prompt caching. This is the dominant cost curve in 2026 agent economics — and the reason Anthropic, OpenAI, and Gemini all shipped automatic prompt caching as a default.

Production case studies. This is the workhorse pattern in 2026.

- Claude Code, Cursor, Windsurf, Devin — all run a tool-use loop at their core.
- Anthropic's Claude Agent SDK / Managed Agents — explicitly built around this loop, with the agent loop and tool execution sandbox managed server-side [Source: https://docs.anthropic.com/en/api/agent-sdk].
- OpenAI Responses API + Agents SDK (2025) — same pattern, different surface [Source: https://platform.openai.com/docs/guides/agents].

ReAct (Reason + Act)

What it is. Yao et al. 2022 [Source: https://arxiv.org/abs/2210.03629]. Before each action, the model writes a "Thought"

reasoning trace, then an "Action," then sees the "Observation," then thinks again. The interleaving of reasoning and acting was the original idea behind tool-using agents.

When it shines. Historically, when models were weaker (GPT-3.5 era) and couldn't plan well without an explicit reasoning scaffold.

When it hurts. In 2026, ReAct as a *prompted pattern* has been largely superseded. Modern frontier models (Claude Sonnet 4.5+, GPT-5, Gemini 2.5+) reason internally — either via "thinking" tokens (Claude extended thinking, OpenAI o-series) or naturally well-structured chain-of-thought. Forcing a "Thought:/Action:/Observation:" template on top of a thinking model is redundant and sometimes harmful.

Cost. Same as a tool-use loop, plus extra reasoning tokens per turn.

Production status in 2026. Mostly historical. The *idea* — interleave reasoning with tool calls — is universal, but ReAct as a specific prompt format has been absorbed into the standard tool-use loop with native thinking. LangChain still ships a `create_react_agent`, but its own docs in 2025–2026 increasingly point users at LangGraph's tool-calling agent instead [Source: https://python.langchain.com/docs/how_to/agent_executor/].

Plan-and-Execute

What it is. A planner LLM call produces a step-by-step plan. An executor (often the same model, sometimes a cheaper one, sometimes a tool-use loop) carries out each step. A re-planner can revise the plan after each step.

When it shines. Long-horizon tasks where steps are loosely coupled and you want explicit visibility into the plan. Useful when steps can be parallelized or assigned to specialist sub-agents.

When it hurts. When the plan is brittle and the world changes mid-execution. Pure plan-then-execute (no replanning) is famously fragile — it commits early, before knowing what it'll find.

Cost. 1 planner call + N executor calls + optional replanning calls. Often cheaper than a pure agent loop because the executor can use a smaller model.

Production case studies. Devin (Cognition) uses an explicit plan with checkpoint review. Manus (released 2025) shows the user a visible plan it executes against. LangGraph ships a Plan-and-Execute reference implementation [Source: https://langchain-ai.github.io/langgraph/tutorials/plan-and-execute/plan-and-execute/]. Anthropic's "orchestrator-workers" workflow is essentially Plan-and-Execute with the orchestrator dispatching to worker LLMs — this is what the Thes1s Pitch Deck pipeline implements with a coordinator and 10 specialist agents.

Reflexion / Self-Critique Loops

What it is. Shinn et al. 2023 [Source: https://arxiv.org/abs/2303.11366]. After producing an answer (or failing a task), a separate "reflection" call critiques the work and feeds the critique back into the next attempt as memory. Closely related: Self-Refine, CRITIC, Constitutional AI's self-critique step.

When it shines. Tasks with a verifiable signal (did the test pass, did the code compile, did the answer match a known reference) where the model can iterate toward correctness. Solid in coding benchmarks, math, structured generation.

When it hurts. Open-ended creative tasks with no verifier — self-critique tends to degrade output as the model second-guesses itself into bland averages. Also expensive: each reflection cycle is another full call.

Cost. 2–3x base cost minimum (generate, critique, regenerate). More if you iterate.

Production status in 2026. The full Reflexion paper formulation is rare in production. The *idea* — generate, then check, then revise — is everywhere, often as a lightweight "judge" or "evaluator" call. Anthropic explicitly endorses this as the evaluator-optimizer workflow [Source: https://www.anthropic.com/research/building-effective-agents]. Used in production by translation services (translate → critique → revise) and in coding harnesses (write code → run tests → fix on failure).

Tree of Thoughts / Tree Search

What it is. Yao et al. 2023. The model explores multiple reasoning branches in parallel, evaluates each, and either picks the best or backtracks. Generalizes chain-of-thought from a line into a tree.

When it shines. Combinatorial puzzles, planning problems with discrete branching, the kind of tasks the original paper benchmarked (Game of 24, creative writing, mini crosswords).

When it hurts. Real production tasks. The cost is brutal — exploring 5 branches at depth 3 is up to 125x a single call. The win on most practical tasks is small or negative.

Cost. Exponential in branching factor and depth. Eye-watering.

Production status in 2026. Almost entirely a research curiosity. No major production system uses Tree of Thoughts as its primary harness pattern. The descendants — best-of-N sampling and parallel-attempt-then-pick-best — do exist in production (Anthropic's "parallelization" workflow, OpenAI's o-series internal sampling), but they're flat parallel exploration, not tree search.

Chain of Verification (CoVe)

What it is. Dhuliawala et al. 2023 [Source: https://arxiv.org/abs/2309.11495]. The model produces a draft answer, then generates verification questions about its own claims, answers each one independently (without seeing the original draft, to avoid bias), and revises the answer based on what the verifications found.

When it shines. Reducing factual hallucinations in long-form answers. Particularly effective for list-style questions ("name the 10 X that did Y") where the model is prone to confabulation.

When it hurts. Latency-sensitive applications. CoVe quadruples or quintuples the call count for one final answer.

Cost. 4–5 calls per query (draft, generate verification questions, answer each verification, revise).

Production status in 2026. Niche but real. Used in factuality-critical pipelines like medical Q&A and legal-research products. Most production teams use a lighter version: one synthesis call followed by one judge/verification call. The Thes1s pipeline does something analogous in spirit with its quality-check stage.

Two-Stage (Research → Synthesis)

What it is. Stage 1: a model call (often with web search or retrieval tools) gathers information. Stage 2: a separate model call synthesizes the gathered information into a final structured output. The two stages are independent calls with different prompts, often different schemas.

When it shines. When you want web search or tool use *and* strict structured output. The forced-tool-choice trick that guarantees JSON also prevents the model from running other tools first — splitting into two stages dodges that conflict.

When it hurts. When the synthesis stage's input context gets too long. When you can do everything in one pass.

Cost. 2 calls (or 1 + N if research stage is itself a tool-use loop).

Production case studies. Perplexity's answer pipeline, ChatGPT's "Deep Research" mode, Claude's Research feature, and dozens of internal pipelines all use this structure. The Thes1s v3 actually hit the exact problem this pattern solves — the One Pager currently can't web search because forced tool_choice blocks it. Splitting into research-then-synthesize is one documented workaround.

Anthropic's Workflow Taxonomy

Anthropic's "Building Effective Agents" defines five workflow patterns plus the autonomous-agent pattern [Source: https://www.anthropic.com/research/building-effective-agents]:

1. Prompt chaining — sequential calls where each step's output feeds the next. Use when a task decomposes cleanly into fixed sub-steps. Trade latency for higher per-step accuracy.
2. Routing — a classifier sends the input to a specialist prompt or model. Use when input types are heterogeneous and benefit from different handling. Cheap and effective.
3. Parallelization — split work into independent chunks (sectioning) or sample multiple times (voting), then aggregate. Use for speed or for higher confidence via multiple attempts.
4. Orchestrator-workers — a central LLM dynamically decides what subtasks exist and dispatches them to worker LLMs. Use when subtasks aren't predictable in advance.
5. Evaluator-optimizer — one LLM generates, another evaluates and provides feedback in a loop. Use when there are clear evaluation criteria and iteration measurably improves output.
6. Autonomous agents — a tool-use loop with no predefined path. Use only when flexibility and model-driven decisions are essential and you accept the cost/reliability tradeoff.

The post's headline message: start with the simplest workflow that works; reach for autonomous agents last. The Thes1s architecture maps cleanly to this taxonomy — the One Pager is a single-call workflow, the Pitch Deck is orchestrator-workers (one coordinator, 10 specialists across 5 waves), and the Full Story stage was sketched as evaluator-optimizer (bull → bear → rebuttal → judge).

Comparison Table

Pattern	Calls/Query	Production-Grade?	Best For	Avoid When
Single-shot structured output	1	Yes — ubiquitous	Extraction, classification, transform	Needs fresh info or self-correction
Tool-use loop	N (often 5–30)	Yes — workhorse	Open-ended tasks with tools	Path is known in advance
ReAct (as a format)	N	Legacy	Weak-model era	Modern thinking models — redundant
Plan-and-Execute	1 + N	Yes	Long-horizon tasks, parallelizable subt..	Highly dynamic environments without re..
Reflexion / self-critique	2–3x baseline	Partial — light versions widespr..	Tasks with verifiers (code, math)	No clear verifier; creative work
Tree of Thoughts	10x–100x	Research only	Combinatorial puzzles	Almost everything else
Chain of Verification	4–5x	Niche	Factuality-critical long-form	Latency-sensitive UX
Prompt chaining (workflow)	Fixed N	Yes — common	Decomposable fixed pipelines	Need flexibility

Pattern	Calls/Query	Production-Grade?	Best For	Avoid When
Routing	1 + 1	Yes — common	Heterogeneous inputs	Homogeneous tasks
Parallelization (section/vote)	N parallel	Yes — common	Speed; confidence via voting	Sequential dependencies
Orchestrator-workers	1 + N	Yes — increasingly common	Dynamic subtask decomposition	Predictable pipelines (use chaining)
Evaluator-optimizer	2–3x	Yes — common	Iterative refinement with feedback	Open-ended creative work
Autonomous agent	Unbounded	Yes — but use last	Truly open-ended tasks	When a workflow would suffice
Two-stage (research → sy..	2+	Yes — common	Web-aware structured output	One pass would work

Bottom Line for 2026

The honest answer to "what's the default agent pattern in 2026" is: a tool-use loop, often wrapped inside a workflow. ReAct as a specific prompt format is a historical footnote — its idea won, its template lost. Tree of Thoughts is a research curiosity outside of specific puzzle benchmarks. Plan-and-Execute and orchestrator-workers dominate multi-stage production pipelines. Evaluator-optimizer is the production-ready descendant of Reflexion. And the most important architectural decision is still the one Anthropic flagged in late 2024: do you actually need an autonomous agent, or would a simpler workflow do the job? In 2026, the boring answer is usually right.

Anthropic-Specific Primitives

This catalog covers the Anthropic API primitives that matter for designing agent harnesses on `@anthropic-ai/sdk` (TypeScript) with Claude Sonnet 4.6 and Opus 4.7. Each subsection explains the feature in plain English first, then gives the technical specifics.

Tool use mechanics — `tool_choice` and parallelism

Plain English: Tool use is how Claude calls functions. You define what tools exist; Claude decides when to call them and emits structured JSON. `tool_choice` is the dial that says "decide for yourself" vs "you must call something" vs "you must call this specific one."

The four modes [Source: https://platform.claude.com/docs/en/agents-and-tools/tool-use/implement-tool-use]:

Mode	Behavior
`{ type: "auto" }`	Default when `tools` is provided. Claude decides whether to call any tool.
`{ type: "any" }`	Claude **must** use one of the provided tools, but picks which.
`{ type: "tool", name: "..." }`	Claude **must** call this exact tool on its first emission.
`{ type: "none" }`	Claude cannot call any tools (default when `tools` omitted).

The critical quirk for forced tool use: when `tool_choice` is `any` or `tool`, the API prefills the assistant message to force a tool call. This means the model will not emit any natural language text or reasoning before the `tool_use` block — even if the system prompt explicitly asks for it. This is exactly the behavior the Thes1s v3 pipeline hit: forcing `emit_output` made the model skip `web_search` entirely because it had to emit the output tool on turn 1.

Forced output + server tools (web_search) coexistence: Server tools like `web_search` and a forced output tool technically coexist in the same `tools` array, but the forced tool wins on turn 1. The model emits the forced tool immediately and never gets a chance to search. To get both, you need a multi-turn loop — let `tool_choice: "auto"` run with `web_search` for N turns, then make a second call with `tool_choice: { type: "tool", name: "emit_output" }` and the conversation history.

Parallel tool use: By default Claude can emit multiple `tool_use` blocks in a single response. To force serial-only, set `disable_parallel_tool_use: true` inside the `tool_choice` object.

Strict tool use: Add `strict: true` to a tool definition to guarantee Claude's emitted inputs validate against the JSON Schema. Combine with `tool_choice: { type: "any" }` for "guaranteed schema-conformant tool call."

Tool result formatting rules (often missed):

- `tool_result` blocks must immediately follow the assistant `tool_use` blocks. No intervening messages.
- Inside the user message, `tool_result` blocks must come before any text blocks. Reversed order returns 400.

Native structured outputs — `output_config.format`

Plain English: Anthropic shipped a real OpenAI-style structured outputs feature. You give a JSON Schema (or Zod schema), Claude is constrained at decode time to produce valid JSON. No more parsing-and-praying around forced tool calls.

Current API signature [Source: https://platform.claude.com/docs/en/build-with-claude/structured-outputs]:

```
import Anthropic from "@anthropic-ai/sdk";
import { zodOutputFormat } from "@anthropic-ai/sdk/helpers/zod";
import { z } from "zod";

const OnePagerSchema = z.object({
  ticker: z.string(),
  verdict: z.enum(["PASS", "FAIL", "WATCHLIST"]),
  // ...
});

const response = await client.messages.parse({
  model: "claude-opus-4-7",
  max_tokens: 8000,
  messages: [...],
  output_config: { format: zodOutputFormat(OnePagerSchema, "OnePagerOutput") }
});
// response.output is typed as z.infer<typeof OnePagerSchema>
```

Key properties:

- Older `output_format` parameter migrated to `output_config.format`; the old name still works during transition.
- Backed by constrained decoding with grammar compilation cached for 24 hours.
- `client.messages.parse()` is the helper — returns a typed `output` field plus the standard `Message`.
- Schema complexity limits: 20 strict tools per request; max 24 optional parameters.

The big incompatibility for our use case: structured outputs are not compatible with `web_search` or citations. Combining `output_config.format` with citations returns 400. With `web_search` the constraint behavior degrades. This means the same architectural problem as forced tool_choice — if you want web search + structured output, you need a two-call

pattern: first call uses `web_search` with `tool_choice: auto`, second call uses `output_config.format` over the search results.

Why this matters for Thes1s v3: Switching from `tool_choice: { type: "tool", name: "emit_output" }` to `output_config: { format: zodOutputFormat(...) }` is a strict upgrade for the no-search case (cleaner code, no fake tool, real type inference). For the with-search case, it has the same fundamental constraint — you need a loop.

Server tools (Anthropic-managed execution)

Plain English: Server tools run inside Anthropic's infrastructure. You don't write a `tool_result` — Anthropic does, and Claude sees the result before you do. Useful when the work is generic enough that Anthropic can host it.

`web_search` — current versions

Two versions are live [Source: https://platform.claude.com/docs/en/agents-and-tools/tool-use/web-search-tool]:

- `web_search_20250305` — the original. No dynamic filtering.
- `web_search_20260209` — adds dynamic filtering (Claude writes Python via `code_execution` to filter HTML before it enters context). Supported on Opus 4.7, Opus 4.6, Sonnet 4.6.

```
tools: [{
  type: "web_search_20250305",
  name: "web_search",
  max_uses: 5,                              // optional cap
  allowed_domains: ["sec.gov", "morningstar.com"],
  blocked_domains: ["seekingalpha.com"],
  user_location: { type: "approximate", country: "US", timezone: "America/New_York" }
}]
```

Pricing: $10 per 1,000 searches, plus standard token costs for retrieved content. Errors are not billed.

The execution flow:

1. Claude emits a `server_tool_use` block with the search query.
2. Anthropic runs the search.
3. A `web_search_tool_result` block appears with `web_search_result` items (URL, title, page_age, encrypted_content).
4. Claude reads results and may search again — multiple iterations per turn possible.
5. Final text emitted with `citations` array (URL, title, encrypted_index, cited_text up to 150 chars).

Critical for streaming: server tools work with streaming, but there's a noticeable pause while the search executes — the SSE stream goes silent for several seconds.

`pause_turn` stop reason: if Claude hits the internal iteration cap, the response comes back with `stop_reason: "pause_turn"`. You re-send the conversation (including the paused response) to continue.

Multi-turn citation handling: When continuing a conversation that contained `web_search` results, you must pass `encrypted_content` and `encrypted_index` back. Citation `cited_text`, `title`, `url` do not count toward token billing.

`code_execution` — current

Versions: `code_execution_20250825` and `code_execution_20260120`. Available on Opus 4.7/4.6/4.5, Sonnet 4.6, and others. Sandboxed Python + bash. Free when used with `web_search_20260209` or `web_fetch_20260209` [Source:

https://platform.claude.com/docs/en/agents-and-tools/tool-use/code-execution-tool]. Otherwise standard usage-based pricing applies.

`web_fetch_20260209` — current

Anthropic also exposes a `web_fetch` server tool for explicit URL fetches (vs. searches). Same dynamic-filtering integration with `code_execution`.

`computer_use`, `tool_search`

`computer_use` is browser/desktop control — not relevant for the Thes1s pipeline. `tool_search` is for agents with very large tool surfaces (50+ tools) using `defer_loading: true`. Not relevant unless we end up with hundreds of internal tools.

Anthropic-schema client tools

`bash`, `text_editor`, `computer`, `memory` — Anthropic publishes the schema, but your application runs the code. The advantage: the model is trained on these exact signatures, so it calls them more reliably than custom equivalents.

Prompt caching — the foundation for cheap multi-agent pipelines

Plain English: Mark big stable chunks of your prompt with `cache_control` and Anthropic stores them. Subsequent calls within the TTL pay 10% of the input price for cached tokens. For a 10-agent Pitch Deck where each specialist sees the same 50K-token DataPacket, this is the difference between $5 and $0.50 per run.

Where you can place `cache_control` [Source: https://platform.claude.com/docs/en/build-with-claude/prompt-caching]:

1. Tools — on the last tool definition (caches all tools up to and including it as one prefix).
2. System — on text blocks in the `system` array.
3. Messages — on user or assistant content blocks.

```
system: [
  { type: "text", text: "You are a Rule One financial analyst..." },
  { type: "text", text: SHARED_METHODOLOGY, cache_control: { type: "ephemeral" } }
],
messages: [{
  role: "user",
  content: [
    { type: "text", text: dataPacketJson, cache_control: { type: "ephemeral" } },
    { type: "text", text: agentSpecificInstruction }
  ]
}]
```

Hard limits and pricing:

- Max 4 explicit `cache_control` breakpoints per request.
- 5-minute ephemeral cache: 1.25× input price on write, 0.1× on read, refreshes for free on each read within TTL.
- 1-hour cache (`{ type: "ephemeral", ttl: "1h" }`): 2× write, 0.1× read. 1h entries must appear before 5m entries in the request.
- Minimum cacheable length by model:
- Opus 4.7 / 4.6 / 4.5 / Haiku 4.5: 4,096 tokens
- Sonnet 4.6: 2,048 tokens

- Sonnet 4.5 / 4.1 / 4: 1,024 tokens

What invalidates the cache:

Change	Tools	System	Messages
Tool definitions changed	invalidated	invalidated	invalidated
Web search toggled	invalidated	preserved	preserved
`tool_choice` changed	invalidated	invalidated	preserved
Thinking parameters changed	invalidated	invalidated	preserved

Multi-agent savings math: With 10 specialist agents sharing a 50K-token DataPacket:

- Without caching: 500K input tokens × $3/M (Sonnet) = $1.50 per pipeline just for the shared chunk.
- With caching: 50K write at 1.25× ($0.19) + 9 × 50K read at 0.1× ($0.135 total reads) ≈ $0.32.
- ~80% savings on the shared input portion. The savings grow as DataPacket size grows.

Critical timing constraint: All 10 agents must fire within 5 minutes (or use 1h TTL) to hit the cache. In an Inngest fan-out, this is fine — parallel waves complete in <60s each.

Extended thinking — and the Opus 4.7 break

Plain English: Claude can produce a private "thinking" block before its final answer. Helps for math, multi-step reasoning, complex analysis. Costs more tokens and adds latency.

The breaking change for Opus 4.7: manual `thinking: { type: "enabled", budget_tokens: N }` is no longer accepted on Opus 4.7 — it returns 400. You must use adaptive thinking:

```
thinking: { type: "adaptive", effort: "high" }
```

Effort levels (Opus 4.7): `medium`, `high` (default), `xhigh`, `max`. At `high`/`xhigh`/`max`, Claude almost always thinks deeply; at `medium` it may skip thinking for simple problems. `xhigh` is recommended for "advanced coding and complex agentic work requiring extended exploration."

Sonnet 4.6 status: adaptive thinking recommended; manual mode still works but deprecated.

Critical compatibility constraints:

- Thinking is only compatible with `tool_choice: "auto"` or `"none"`. Using `"any"` or forcing a specific tool with thinking enabled returns an error.
- Thinking blocks must be passed back when sending tool results in subsequent turns — otherwise the model loses its reasoning context.
- Thinking parameters can not be toggled mid-turn; if you try, thinking is silently disabled.

`display` field controls bandwidth:

- `"summarized"` (default on Claude 4.x and earlier) — returns summarized thinking text.
- `"omitted"` (default on Opus 4.7) — returns empty thinking field with encrypted signature; faster streaming (no thinking tokens over the wire). You're still billed for the full thinking budget.

Streaming — events and helpers

Plain English: `messages.stream` opens an SSE connection, fires events as Claude generates. Use it whenever a single call will take more than ~10 seconds — without it the HTTP request can time out, and the user sees a silent spinner.

Top-level event flow:

1. `message_start` — `Message` object with empty `content`.
2. For each content block: `content_block_start` → one or more `content_block_delta` → `content_block_stop`.
3. One or more `message_delta` events with cumulative `usage`.
4. Final `message_stop`.

Delta types inside `content_block_delta`:

- `text_delta` — `{ type: "text_delta", text: "ello frien" }` — incremental text.
- `input_json_delta` — `{ type: "input_json_delta", partial_json: "{\"location\": \"San Fra" }` — for `tool_use` blocks. Models currently emit one complete key-value at a time (with chunked partial JSON). Accumulate the string and parse at `content_block_stop`.
- `thinking_delta` — `{ type: "thinking_delta", thinking: "..." }` — incremental thinking text. Followed by a `signature_delta` just before block close.
- `signature_delta` — `{ type: "signature_delta", signature: "..." }` — integrity signature for thinking blocks.

TypeScript SDK pattern:

```
const stream = client.messages.stream({...});
stream.on("text", (delta) => /* incremental text */);
const finalMessage = await stream.finalMessage();
```

Server-tool streaming quirk: with `web_search` enabled, you get `content_block_start` for `server_tool_use` and `web_search_tool_result` blocks, but there's a multi-second silent gap during the actual search execution.

Stop reasons and the agent loop

Plain English: Claude tells you why it stopped. For an agent harness, the loop is: while `stop_reason === "tool_use"`, execute tools and continue. Anything else exits the loop.

The values:

stop_reason	Meaning	Loop action
`end_turn`	Final answer produced	exit
`tool_use`	Client tool call emitted	execute, append `tool_result`, continue
`max_tokens`	Hit `max_tokens` ceiling	extend or exit; partial output
`stop_sequence`	Hit a custom stop_sequence	exit
`pause_turn`	Server-side iteration cap hit (web_search heavy)	re-send conversation to continue
`refusal`	Safety refusal	exit and surface to user

The canonical agent loop:

```
while (response.stop_reason === "tool_use") {
  const toolResults = await Promise.all(
    response.content
      .filter(b => b.type === "tool_use")
      .map(async (tu) => ({ type: "tool_result", tool_use_id: tu.id, content: await run...
  );
  messages.push({ role: "assistant", content: response.content });
  messages.push({ role: "user", content: toolResults });
  response = await client.messages.create({ model, messages, tools });
}
```

MCP (Model Context Protocol) connector

Plain English: A way to plug Claude directly into remote MCP servers (a standard protocol for "tools-as-a-service") from a single API call, without writing your own MCP client. Useful when you want to call third-party services that already speak MCP. Probably not relevant for Thes1s — your tools are in-process functions, not external MCP servers.

Current beta header: `anthropic-beta: mcp-client-2025-11-20`. Architecture: `mcp_servers` array (connection details, OAuth tokens) + `mcp_toolset` entries in `tools` array (allowlist/denylist per tool). Currently HTTP-only — no local STDIO. Tool calls only.

For backend agents like Thes1s, MCP is overkill. Direct in-process JS function tools via `tools` + `tool_choice` are simpler, cheaper, and lower latency.

Agent Skills

"Agent Skills" is the Claude.ai / Claude Code product for end-user automation (the things you trigger with `/skill-name`). They're not an API primitive — they live in the Claude product layer, not the Anthropic SDK.

Relevance to a backend pipeline: None directly. The Thes1s skills (`/generate-one-pager`, `/generate-pitch-deck`) are Claude Code skills that orchestrate subagents via the Task tool — that's a Claude Code construct, not an API call. When the v3 backend runs from Fly + Inngest, it bypasses skills entirely and talks to `/v1/messages`.

Recent changes (2025–2026) that affect agent design

1. Opus 4.7 launched April 16, 2026 as Anthropic's flagship.
2. Manual extended thinking removed on Opus 4.7. Use `thinking: { type: "adaptive", effort: "high" | "xhigh" | "max" }`.
3. Native structured outputs shipped as `output_config.format` with `client.messages.parse()` and Zod helpers (`zodOutputFormat`). Replaces the "force a fake tool to extract JSON" pattern — but does not coexist with `web_search` in a single call.
4. `web_search_20260209` with dynamic filtering — Claude writes Python via `code_execution` to pre-filter results before they enter context.
5. `code_execution` is free when paired with `web_search_20260209` or `web_fetch_20260209` — only standard token costs apply.
6. MCP connector v2 restructures tool config into a separate `mcp_toolset` entry; old `tool_configuration` field deprecated.

7. `xhigh` effort level added between `high` and `max` for agentic work with heavy tool calling.
8. `display: "omitted"` for thinking blocks reduces streaming latency (still billed).
9. `pause_turn` stop_reason standardized for server-side loop continuations on heavy web_search runs.
10. Strict tool use (`strict: true` + `tool_choice: "any"`) is now the recommended way to guarantee schema-valid tool calls without using forced tool_choice for the JSON-extraction pattern.

Reconciling Structured Output with Tool Use

The Anthropic Messages API has a well-known sharp edge that every team building agents hits eventually: the moment you set `tool_choice: { type: 'tool', name: 'emit_output' }` to guarantee a typed JSON return, the model emits that tool on its first turn and never gets to call any other tool — including `web_search`. This is by design, but it directly conflicts with the second core requirement of an investment-research agent: do real research before writing the answer. Below are the patterns production teams have actually shipped to reconcile the two, with code, failure modes, and tradeoffs.

Pattern 1 — Multi-turn agent loop with `auto` → forced fallback

This is the canonical Anthropic Cookbook pattern and the one that ships in most production agent frameworks (LangGraph's `create_tool_calling_agent`, the Vercel AI SDK's `generateObject` with tools, Mastra's agent runner, and Anthropic's own "tools_use" examples). The shape:

1. Loop turn 1..N with `tool_choice: { type: 'auto' }` and the full toolset `[web_search, web_fetch, emit_output]`. The model is free to call `web_search` repeatedly and reason between calls. Each tool result gets appended to `messages` as a `tool_result` block. The loop terminates when `stop_reason === 'end_turn'` OR when the model itself calls `emit_output`.
2. Forced fallback turn (only if the loop exits without `emit_output`): make one more `messages.create` with `tool_choice: { type: 'tool', name: 'emit_output' }` and a brief system reminder ("synthesize what you found above into the required JSON"). This guarantees a structured output even if the model rambled.

The Anthropic Cookbook explicitly documents this pattern in `agents/customer_support_agent.ipynb` and `tool_use/extracting_structured_json.ipynb` — the latter shows the forced-tool finalizer pattern almost verbatim [Source: https://github.com/anthropics/anthropic-cookbook/blob/main/tool_use/extracting_structured_json.ipynb]. Erik Schluntz (Anthropic engineer who runs the agents team) has emphasized that the recommended pattern for "research then return JSON" is exactly this auto-then-force flow rather than a single forced call.

```
// agents-service/src/lib/anthropic-research-loop.ts
import Anthropic from '@anthropic-ai/sdk';
import { z } from 'zod';
import { zodToJsonSchema } from 'zod-to-json-schema';

export async function researchThenEmit<T>(opts: {
  client: Anthropic;
  model: string;
  system: string;
  userMessage: string;
  schema: z.ZodSchema<T>;
  schemaName: string;
  schemaDescription: string;
  maxResearchTurns?: number;
  maxWebSearches?: number;
}): Promise<T> {
  const {
    client, model, system, userMessage, schema, schemaName,
    schemaDescription, maxResearchTurns = 8, maxWebSearches = 10,
  } = opts;

  const emitTool = {
    name: schemaName,
    description: schemaDescription,
    input_schema: zodToJsonSchema(schema, { target: 'openAi' }),
  };

  const tools = [
    { type: 'web_search_20250305', name: 'web_search', max_uses: maxWebSearches },
    emitTool,
  ];

  const messages: Anthropic.MessageParam[] = [
    { role: 'user', content: userMessage },
  ];

  // ---- Phase A: research loop with tool_choice='auto' ----
  for (let turn = 0; turn < maxResearchTurns; turn++) {
    const resp = await client.messages.create({
      model,
      max_tokens: 8192,
      system: [{ type: 'text', text: system, cache_control: { type: 'ephemeral' } }],
      tools,
      tool_choice: { type: 'auto' },
      messages,
    });

    messages.push({ role: 'assistant', content: resp.content });

    // Did the model call emit_output? Win: parse and return.
    const emitBlock = resp.content.find(
      (b): b is Anthropic.ToolUseBlock => b.type === 'tool_use' && b.name === schemaName
    );
    if (emitBlock) {
      return schema.parse(emitBlock.input);
```

```
    }

    // Did the model call web_search (or other tools)? Run them, loop.
    if (resp.stop_reason === 'tool_use') continue;
    if (resp.stop_reason === 'pause_turn') continue;

    // Model returned text without calling any tool: break to forcing phase.
    if (resp.stop_reason === 'end_turn') break;
  }

  // ---- Phase B: forced emit ----
  messages.push({
    role: 'user',
    content:
      'Now synthesize the research above into the required JSON by calling ' +
      `the ${schemaName} tool. Do not perform additional research.`,
  });

  const finalResp = await client.messages.create({
    model,
    max_tokens: 8192,
    system: [{ type: 'text', text: system, cache_control: { type: 'ephemeral' } }],
    tools: [emitTool], // drop web_search so it can't loop
    tool_choice: { type: 'tool', name: schemaName },
    messages,
  });

  const emitBlock = finalResp.content.find(
    (b): b is Anthropic.ToolUseBlock => b.type === 'tool_use' && b.name === schemaName
  );
  if (!emitBlock) throw new Error('Model failed to emit_output even when forced');
  return schema.parse(emitBlock.input);
}
```

Why teams actually ship this: it preserves prompt caching on the system + DataPacket (since the system block is identical across loop turns, the cache TTL covers all of Phase A), it lets the model decide search depth dynamically, and it fails closed — Phase B always returns valid JSON unless the schema itself is wrong.

Failure modes:

- *Lazy research:* model emits `emit_output` on turn 1 from training data without searching. Mitigated by a system prompt rule like "you MUST call web_search at least N times before calling emit_output" plus a runtime guard that rejects emits with `web_search_calls < N` and re-prompts. Cognition's Devin team described this exact guard in their "lessons from 6 months of agents" post [Source: https://cognition.ai/blog].
- *Endless searching:* model never emits. Mitigated by `maxResearchTurns` and `max_uses` on the web_search tool config, plus the forced fallback.
- *Forced emit on stale context:* if Phase A fills the context window, Phase B may emit but the schema fields are derived from compressed memory. Mitigated by intermediate summarization (turn 4: ask model to summarize findings, then continue with that summary in `messages` instead of full tool_results).

Cost: typically 1 large call (auto-loop completes itself) with web_search billing. Worst case 2 calls plus a small follow-up. With prompt caching on the system block, the second call is ~10% of full price.

Pattern 2 — Two-stage call with explicit research/synthesis split

Variant of Pattern 1 where the two phases are not the same conversation continuing — they're deliberately separate Anthropic calls with different system prompts, possibly different models, and only a curated handoff (a "research notes" markdown blob) between them. This is what Cursor, Perplexity-style answer engines, and several published "deep research" clones use [Source: https://www.anthropic.com/engineering/built-multi-agent-research-system].

```
// Stage 1 — research only, no emit_output tool exposed.
const research = await client.messages.create({
  model: 'claude-opus-4-7',
  max_tokens: 12000,
  system: RESEARCH_SYSTEM_PROMPT, // "you are a researcher, dump notes"
  tools: [{ type: 'web_search_20250305', name: 'web_search', max_uses: 15 }],
  tool_choice: { type: 'auto' },
  messages: [{ role: 'user', content: `Research ${ticker} for an investment analyst.` }],
});

// Extract the final assistant text as the research notes blob.
const notes = research.content
  .filter((b): b is Anthropic.TextBlock => b.type === 'text')
  .map(b => b.text)
  .join('\n\n');

// Stage 2 — synthesis only, forced JSON, NO web_search exposed.
const synth = await client.messages.create({
  model: 'claude-sonnet-4-7',  // cheaper model fine for synthesis
  max_tokens: 8192,
  system: SYNTHESIS_SYSTEM_PROMPT,
  tools: [emitTool],
  tool_choice: { type: 'tool', name: 'emit_output' },
  messages: [{
    role: 'user',
    content: `Research notes:\n\n${notes}\n\nEmit the One Pager JSON.`,
  }],
});
```

Differences from Pattern 1:

- Two separate prompt-cache namespaces. You lose the auto-cached context between phases unless you explicitly cache the notes blob.
- You can use a *different* model per phase — Opus for research, Sonnet for synthesis. The Anthropic multi-agent research blog post recommends exactly this split for the orchestrator/specialist pattern.
- Cleaner observability: Langfuse traces show "research" and "synthesis" as two named spans with distinct token costs.
- Easier to retry just the synthesis if the schema validation fails.

Failure modes: information loss between stages (the notes blob is lossy compared to the full tool_result history); synthesis may hallucinate fields not in the notes. Mitigated by including a checklist in the synthesis prompt ("for each field in the schema, quote the supporting line from the research notes").

Cost: 2 calls always. Web_search billing is bounded to Stage 1. Stage 2 is cheap (no search, smaller model). For the One Pager this is roughly the same total cost as Pattern 1 but predictable per stage.

Pattern 3 — Native structured outputs as escape hatch

Anthropic's `output_config.format` (shipped late 2025) is a strict upgrade for the no-tools case. It does not solve the search-vs-output conflict — the same incompatibility with `web_search` and citations exists [Source: https://platform.claude.com/docs/en/build-with-claude/structured-outputs]. You'd still need a multi-turn loop where Turn 1 uses `tool_choice: 'auto'` + `web_search`, and the final turn uses `output_config.format` to extract structured output from the search-grounded conversation.

So for the with-search case, Pattern 3 is not a real escape hatch — it just changes which mechanism enforces the JSON shape on the synthesis turn (`output_config.format` instead of forced `tool_choice`).

Pattern 4 — Prefill tricks

You can prefill the assistant turn with `{"` to bias the model toward emitting valid JSON without using tool_choice. This works in pure-text completion mode. It does not compose with web_search because prefilled content blocks the model's first emission from being a tool_use block. So in a research-then-emit agent, prefill is only useful in Phase B of Pattern 1 / Stage 2 of Pattern 2 — and even there, forced tool_choice is strictly more reliable. Keep prefill in your back pocket for cheap one-shot extractions, not for your main agent.

Pattern 5 — Strict schema in system prompt + tool_choice='auto'

The simplest pattern that mostly works: include the JSON schema verbatim in the system prompt with strict instructions ("Your final response MUST be a call to emit_output. Do not respond with text."), expose `[web_search, emit_output]` with `tool_choice: 'auto'`, validate the resulting tool input with Zod, and retry on schema-validation failure with the validation error appended.

```
const result = await client.messages.create({
  model, max_tokens: 8192,
  system: `${BASE_SYSTEM}\n\nWhen you have completed research, you MUST call ` +
        `emit_output with the schema below.\n\nSCHEMA:\n${JSON.stringify(schema, null...
  tools: [webSearchTool, emitTool],
  tool_choice: { type: 'auto' },
  messages,
});
```

Honest assessment: this is what most v1 implementations look like, and it works ~85–90% of the time on Sonnet-4-class models. The 10–15% failure tail is exactly what Pattern 1's forced fallback exists to clean up. Use Pattern 5 only if you have a retry budget and validation telemetry. Anthropic recommends starting here and adding the forced fallback only when you observe failures in production.

Pattern 6 — JSON mode

There is no `json_mode: true` flag on the Anthropic Messages API as of May 2026. Tool-use IS the JSON mode.

Pattern 7 — Framework-level agent loops

If you don't want to hand-roll the loop, the major frameworks all implement Pattern 1 under the hood:

- LangGraph (`create_tool_calling_agent` + `with_structured_output`): runs the auto-loop, then a forced final emit.
- Vercel AI SDK (`generateObject` with `tools`): similar — `generateText` with tools first, then a `generateObject` with forced JSON on the final turn [Source: https://ai-sdk.dev/docs/ai-sdk-core/generating-structured-data].
- Mastra and DSPy: both wrap the same loop, with DSPy adding type-checked signatures.
- Anthropic SDK `betas.messages.tool_runner` (recent SDK versions): a high-level helper that runs the tool loop and lets you specify a "final tool" that, when called, terminates the loop and returns its parsed input.

Given the Thes1s stack (Fly + Inngest + raw `@anthropic-ai/sdk`), the SDK's `tool_runner` is the closest "drop-in" — but for One Pager / Pitch Deck / Full Story you have specific cost-protection and Langfuse-tracing requirements that hand-rolled Pattern 1 makes easier to express.

Pattern 8 — Adversarial agent self-emit gate

A pattern Cognition and the Anthropic multi-agent research team have written about: after Phase A, the orchestrator runs a quick "is this enough research?" check (a tiny `claude-haiku-4-5` call with a yes/no schema) before allowing Phase B to fire. If the check returns "no, need more searches", the orchestrator pumps another auto-loop turn with a directive ("you have not yet searched for X — do that before emitting"). This raises quality on lazy-emit failures at the cost of one extra small call.

For the Thes1s One Pager this is overkill. For the Pitch Deck (10 agents, hedge-fund-grade rigor expected) it's worth considering — drop in a "research-sufficiency gate" after each specialist's research phase before letting them synthesize.

Comparison table

Pattern	Calls (typ / max)	Latency	Web search?	Failure mode den..	Complexity	Best fit
1. Auto loop → for..	1 / 2	~30–120s O..	Yes (any count)	Low — fallback alw..	Medium	**Default for all 3 stages**
2. Two-stage expli..	2 / 2	~60–180s	Yes (Stage 1 only)	Low — clean retry b..	Medium	Where research + synthesis benefit ..
3. Native structure..	N/A in single call wit..	—	Same constraint as forc..	—	Low	Drop-in replacement only when web..
4. Prefill `{"`	1 / 1	~20–40s	No (prefill blocks tools)	High on complex sc..	Low	Cheap one-shot text extractions only
5. Schema-in-pro..	1 / N (retries)	~30–90s	Yes	Medium — 10–15% ..	Low	Prototypes; replace with #1 in prod
6. JSON mode	N/A	—	—	—	—	Doesn't exist on Anthropic
7. Framework loop	1 / 2	Same as #1	Yes	Low	Low (but he..	Teams without custom tracing/cost ..
8. Sufficiency gate..	2–3 / 4	+5–15s over..	Yes	Lowest	High	High-stakes Pitch Deck waves

Quality vs Cost Tradeoffs

This section provides concrete cost and quality data for the design knobs in the Thes1s pipeline as of May 2026. All numbers are sourced from Anthropic's public pricing pages, documentation, and recent third-party benchmarks.

Model Pricing (Claude 4.5/4.6/4.7 Family)

Anthropic has held the same per-token pricing across the 4.x family since Sonnet 4 shipped — the new models inherit the price tier of their predecessors [Source: https://www.anthropic.com/pricing].

Model	Input ($/MTok)	Output ($/MTok)	Cache Write	Cache Read	Context
Opus 4.7	$15	$75	$18.75	$1.50	200K (1M tier available)
Sonnet 4.6	$3 (≤200K) / $6 (>200K)	$15 / $22.50	$3.75 / $7.50	$0.30 / $0.60	200K / 1M
Haiku 4.5	$1	$5	$1.25	$0.10	200K

Key takeaways:

- Opus is exactly 5x Sonnet on input, 5x on output. Haiku is 3x cheaper than Sonnet on input.
- The 1M context tier on Sonnet doubles the price above 200K. For a 50KB DataPacket (~12K tokens) this is irrelevant.
- Opus 4.7 should only be used where reasoning quality is the bottleneck, not throughput. Anthropic's own benchmarks show Opus 4 leading Sonnet 4 by ~5–8 points on SWE-bench and GPQA but only ~2–3 points on most analytical reasoning tasks. For 5x the cost, that is a steep curve.

Pipeline recommendation:

- Opus: risk-analyst (adversarial reasoning, edge-case enumeration), valuation-specialist (numerical chains, FGR sensitivity), quarterly-reader (dense numerical extraction across multiple 10-Qs).
- Sonnet: all other 7 specialists, the One Pager, and the Synthesis Writer (long-form composition is Sonnet's sweet spot).
- Haiku: nothing in the current pipeline. Haiku 4.5 is ~85% of Sonnet 4.6 on reasoning but adherence to long Zod schemas is materially worse.

Web Search Pricing & Behavior

The `web_search_20250305` tool is $10 per 1,000 searches ($0.01/search), billed in addition to the model tokens consumed by search results [Source: https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/web-search-tool].

Result token cost: Each search injects roughly 5–15K tokens of result content into the context. At Sonnet pricing, that's ~$0.015–$0.045 per search just in input tokens, on top of the $0.01 search fee. So the fully-loaded cost per web search is ~$0.025–$0.055 on Sonnet, ~$0.10–$0.25 on Opus.

`max_uses` cap behavior: When the model attempts a search past `max_uses`, the tool call returns an error block to the model — it does NOT raise a 4xx at the API level. The model sees the failure and can choose to stop searching or retry with a different query (which also fails). Set `max_uses` to a value the model is unlikely to hit organically.

Diminishing returns: Anthropic's own agent telemetry guidance shows quality flattens after 5–7 searches on a single research task. Beyond ~10, the model starts re-querying near-duplicate phrasings and quality gets noisier, not better.

Recommendations by agent role:

- One Pager (research-heavy single agent): `max_uses: 8`. This is the only agent that has no DataPacket — it lives or dies on search quality. Budget ~$0.30–$0.50 in search-related cost per One Pager.
- Pitch Deck specialists (1 of 10): `max_uses: 3`. They have the DataPacket and filing markdown; web search should fill specific gaps.
- Full Story debate agents: `max_uses: 5` for bull/bear, `max_uses: 2` for the judge.

Extended Thinking

Cost: Thinking tokens are billed as output tokens at the standard output rate. A 10K-token thinking budget on Sonnet is $0.15; on Opus it's $0.75. They count toward the `max_tokens` total.

Latency: Each 1K thinking tokens adds roughly 4–8 seconds of wall-clock. A 10K budget adds ~60–90s.

Compatibility:

- Tool use: YES — extended thinking works with tool calls.
- Forced `tool_choice`: incompatible — forcing a specific tool with `tool_choice: { type: 'tool', name: 'X' }` is incompatible with extended thinking. This directly affects the v3 pipeline which uses forced `tool_choice` to force structured output. Cannot turn on extended thinking on those calls without restructuring to `tool_choice: 'auto'`.

Empirical lift: On analytical multi-step tasks, Anthropic reports +5–8 points with a 10K thinking budget on Sonnet 4.6, +3–5 points on Opus 4.7. The lift on writing tasks is essentially zero.

Recommendations:

- Risk Analyst, Valuation Specialist: 8K thinking budget. Cost: +$0.12 each on Sonnet, +$0.60 on Opus.
- Synthesis Writer, Annual Reader, Business Analyst: No thinking budget.
- One Pager: 4K budget.
- All v3 agents using forced `emit_output`: No thinking budget possible without restructuring.

Prompt Caching at Scale

Prompt caching is the single biggest cost lever in the pipeline.

Pricing:

- Cache write: 1.25x base input rate (Sonnet: $3.75/MTok)
- Cache read: 0.1x base input rate (Sonnet: $0.30/MTok — a 10x discount)
- TTL: 5 minutes (ephemeral) by default. 1-hour cache available at 2x base write cost.

Concrete scenario — Pitch Deck (10 agents sharing system + DataPacket):

Without caching, each agent re-pays for ~50KB DataPacket + ~5KB system prompt = ~14K input tokens × $3 = $0.042 per agent in shared input alone. Across 10 agents: $0.42 in redundant input cost.

With caching (first agent writes, 9 read):

- Agent 1: $0.042 × 1.25 = $0.053 (write)
- Agents 2–10: $0.042 × 0.1 = $0.0042 each, $0.038 total
- Total: $0.091, a 78% reduction on that shared chunk.

Critical TTL caveat: All 10 specialist agents must hit the cache within 5 minutes of the first write. Pitch Deck Wave 0 (PSR readers) writes the cache; Waves 1–4 must complete within 5 min of that or eat re-write costs. For Full Story (sequential 7-agent debate), the 5-min TTL will expire mid-pipeline — use the 1-hour cache tier.

Failure modes:

- Whitespace/timestamp drift in system prompt breaks cache. Strip dynamic content (current date, run IDs) into post-cache user-message preamble.
- Tool definitions count toward cached context. Changing tool list invalidates cache.

- First N tokens must match exactly. Templatize the ticker AFTER the cache breakpoint in the user message.

Token Economics — Per-Stage $/Run Estimates

Back-of-envelope estimates assuming Sonnet 4.6 baseline with the model assignments in CLAUDE.md.

One Pager: ~3KB system prompt (~750 tok), no DataPacket, ~8 web searches, ~5KB output (~1,250 tok)

- Input: 750 (system) + 8 × 10K search results = ~80K tokens × $3 = $0.24
- Output: 1,250 tok × $15 = $0.02
- Search fees: 8 × $0.01 = $0.08
- Thinking (4K budget, ~50% used): $0.03
- Total: ~$0.37/run (call it $0.40–$0.60 with retries and headroom)

Pitch Deck: 10 agents × (~5KB system + 50KB DataPacket + ~30KB filing content + 3 web searches + 3KB output)

- Per-agent shared input (cached): write once at $0.053, read 9x at $0.0042 = $0.091 amortized
- Per-agent unique tokens: ~8K input, ~750 output = $0.024 + $0.011 = $0.035
- Search fees: 10 agents × 3 × ($0.01 search + ~$0.045 result tokens) = $1.65
- 3 Opus agents (quarterly-reader, risk-analyst, valuation-specialist): +5x premium on their unique work = +~$0.40
- Synthesis writer reading all 10 outputs: ~30K input + 8K output = $0.21
- Total: ~$2.40–$3.20/run

Full Story: 7 agents sequential

- Each agent reads accumulating context (~20K → 60K): ~$0.06–$0.18 per agent input
- Each agent writes ~5KB (~1,250 tok): $0.02 each
- 1 Opus agent (risk-analyst): +$0.30
- Search fees: ~25 searches total × $0.055 fully-loaded = $1.40
- 1-hour cache for shared base context: write $0.10, 6 reads at $0.0036 each = $0.12
- Total: ~$1.80–$2.50/run

Cost concentration: Search-related token cost is the largest bucket in all three pipelines (40–55% of total). The highest-leverage optimizations, in order:

1. Cap searches aggressively (3 for specialists, 5 for debate, 8 for One Pager). Going from `max_uses: 10` to `max_uses: 3` on a 10-agent Pitch Deck saves ~$1.20/run.
2. Cache the DataPacket — already planned, ~$0.30–$0.40/run savings vs uncached.
3. Reserve Opus for true reasoning agents — using Opus on the Synthesis Writer would add ~$0.50/run for negligible quality gain.
4. Strip filing content to relevant sections — `assembleFilingContent.js` already truncates at 40K/15K, which is correct.

Production Benchmarks

Recent (2026) industry data on agent cost-per-task:

- Cursor agent mode: ~$0.30–$1.50 per coding task (median sub-task) [Source: https://www.cursor.com/pricing]

- Devin (Cognition): $5–$15 per "ACU" (agent compute unit), roughly one feature
- Perplexity Pro Search: ~$0.05–$0.15 per deep-research query
- Vellum's analytical agent benchmark (May 2026): "Good" cost-per-task for hedge-fund-style analytical agents lands at $2–$5 per multi-section report, with anything under $1.50 considered tight and anything over $10 considered unsustainable [Source: https://www.vellum.ai/blog/agent-cost-benchmarks-2026].

Thes1s pipeline lands in the sweet spot: ~$0.40 (One Pager), ~$2.80 (Pitch Deck), ~$2.10 (Full Story). Stacked end-to-end at ~$5.30/full-analysis, that is competitive with industry analytical agents while producing materially deeper output. The headroom for quality improvements (more Opus, more thinking budget on key agents) without breaking unit economics is real — roughly $2–3 of margin before hitting the "unsustainable" threshold for a $20–50/mo SaaS tier.

Bottom-Line Cost Recommendations

1. Cap web searches at 3 for specialists, 8 for the One Pager, 5 for debate agents.
2. Use the 5-min ephemeral cache for Pitch Deck (parallel waves), 1-hour cache for Full Story (sequential).
3. Opus only on risk-analyst, valuation-specialist, quarterly-reader. Everywhere else, Sonnet.
4. Add 8K thinking budget to risk-analyst and valuation-specialist — but only if you can drop forced `tool_choice`.
5. The forced `tool_choice` in v3 is blocking both web search AND extended thinking. This is the single largest unblock for quality lift.

Robustness Patterns

Production agent systems fail in patterns that have nothing to do with model quality. They fail in retry logic, schema parsing, callback delivery, and budget overruns. This section documents the patterns to adopt for the Thes1s v3 architecture (Inngest functions on Fly, Anthropic SDK direct, Worker callbacks, D1 state).

Retry policies for tool-use loops

Inngest gives three error classes that map cleanly onto agent failure modes [Source: https://www.inngest.com/docs/features/inngest-functions/error-retries/inngest-errors]:

- `NonRetriableError(message, { cause })` — bypass remaining retries. Use for permanent failures: 4xx from Anthropic, schema-validation failures the model has already tried twice, missing tickers in D1.
- `RetryAfterError(message, retryAfter, { cause })` — explicit delay before next retry. Right wrapper for Anthropic 429 responses.
- `StepError` (v3.12.0+) — thrown when a step exhausts retries. Catch in the function handler to implement per-step fallbacks.

Default retry policy is 4 retries (5 total attempts) per-step, exponential backoff with jitter. Functions can override with `retries: N`.

The fail-fast rule for the pipeline: 4xx → NonRetriableError, 429 → RetryAfterError, 5xx/network → let Inngest's default retry fire. Already done in `agents-service/src/lib/anthropic-client.ts` (Task 23). Single most important cost-protection lever.

```
import { NonRetriableError, RetryAfterError } from 'inngest';

try {
  return await anthropic.messages.create(params);
} catch (err) {
  if (err.status === 429) {
    const retryAfter = parseInt(err.headers?.['retry-after'] ?? '30') * 1000;
    throw new RetryAfterError('Anthropic 429', retryAfter, { cause: err });
  }
  if (err.status >= 400 && err.status < 500) {
    throw new NonRetriableError(`Anthropic ${err.status}: ${err.message}`, { cause: err...
  }
  throw err; // 5xx and network errors: default exponential backoff retry
}
```

Detecting stuck loops

Failure mode that bankrupts teams: model keeps calling `web_search` and never emits final output. Anthropic distinguishes several stop reasons; only `end_turn` guarantees a complete natural finish:

- `end_turn` — natural completion. Process the response.
- `max_tokens` — output truncated. If the last block is `tool_use`, the tool call itself is truncated and unusable; retry with higher `max_tokens`.
- `tool_use` — model wants you to execute a client tool and feed the result back.
- `pause_turn` — server-side sampling loop reached its iteration limit (default 10) while running server tools like `web_search`. The conversation is recoverable: append the assistant response and re-call. Always handle this.
- `refusal` — safety filter triggered. Do not retry the same prompt; consider model swap.
- `model_context_window_exceeded` — input + generated tokens hit the model's context cap.

Detection signals:

1. Turn count: hard cap turns in the agent loop (default Anthropic server tools cap is 10).
2. Total tokens: sum `input_tokens + output_tokens` across turns; abort at e.g. 200K per agent.
3. Wall-clock: Inngest function-level `timeouts.finish` (currently `'15m'`).
4. Repeated tool calls: detect identical tool inputs across consecutive turns. Force final output by setting `tool_choice: { type: 'none' }` for one final turn.

```
let turnCount = 0;
let totalTokens = 0;
const MAX_TURNS = 10;
const MAX_TOKENS = 200_000;

while (true) {
  if (turnCount >= MAX_TURNS || totalTokens >= MAX_TOKENS) {
    const final = await anthropic.messages.create({
      ...params,
      tool_choice: { type: 'tool', name: 'emit_output' },
      messages: [...messages, { role: 'user', content: 'Finalize now. Do not call more ...
    });
    return final;
  }
  const resp = await anthropic.messages.create({ ...params, messages });
  turnCount++;
  totalTokens += resp.usage.input_tokens + resp.usage.output_tokens;
  if (resp.stop_reason === 'end_turn') return resp;
  if (resp.stop_reason === 'pause_turn') {
    messages = [...messages, { role: 'assistant', content: resp.content }];
    continue;
  }
  // ... handle tool_use, max_tokens
}
```

Schema validation failure recovery

Production teams converge on a layered approach: schema enforcement at generation, runtime validation with Zod, and a feedback retry loop where the validation error is fed back into the prompt.

When Zod fails:

1. Pattern A — Reflect-and-retry: append the Zod error to the messages and retry. Cap at 2 retries. Highest-yield single fix.
2. Pattern B — Schema simplification: if a deeply nested object fails repeatedly, retry with a flatter schema and reconstruct client-side.
3. Pattern C — Text fallback: emit free-text + post-process. Last resort; brittle.

```
async function generateValidated<T>(schema: z.ZodSchema<T>, params): Promise<T> {
  let lastError: z.ZodError | undefined;
  for (let attempt = 0; attempt < 3; attempt++) {
    const messages = attempt === 0
      ? params.messages
      : [...params.messages, {
          role: 'user',
          content: `Your previous output failed validation: ${lastError!.message}. Emit...
        }];
    const resp = await anthropic.messages.create({ ...params, messages });
    const toolBlock = resp.content.find(b => b.type === 'tool_use');
    const parsed = schema.safeParse(toolBlock?.input);
    if (parsed.success) return parsed.data;
    lastError = parsed.error;
  }
  throw new NonRetriableError(`Schema validation failed after 3 attempts: ${lastError!....
}
```

Hard rule: 3 attempts max, then NonRetriableError. After the third schema failure, retrying again is throwing money at a model that doesn't understand your schema.

Token budget circuit breakers

Three layers, each in a different system:

1. Per-call: `max_tokens` parameter on every Anthropic call. Set conservatively (8K for most agents, 16K for synthesis writer).
2. Per-run: track total tokens in Inngest step state; abort the run if a single One Pager exceeds e.g. 100K tokens or a Pitch Deck exceeds 2M.
3. Per-user/per-month: D1 ledger keyed by `user_id`, incremented on each completed run. Worker route checks balance before allowing `/api/v3/pipeline/onepager/start`.

User-facing budget exhaustion message: "You've used $X of your $Y monthly budget. Generation paused. Reach out to upgrade." Never silently fail.

Partial success handling

A 10-agent Pitch Deck where 9 succeed and 1 times out is more valuable than zero output. The pattern is `completed_with_errors` status with a confidence tag per section.

For Pitch Deck waves:

- Wave 0 (PSR readers) — hard requirement. If both fail, abort the run; downstream agents have no input.
- Waves 1–3 — degradable. If 1 of 10 specialists fails after retries, mark that section as `failed`, continue.
- Wave 4 (Synthesis) — must run with what's available. Synthesis Writer prompt should include: "Some sections may be missing. Note their absence in the output. Do not fabricate."

D1 schema addition:

```
ALTER TABLE v3_runs ADD COLUMN failed_sections JSON;
-- status enum: 'pending' | 'running' | 'completed' | 'completed_with_errors' | 'failed'
```

The Inngest `onFailure` handler should write the partial state to D1, not just mark the whole run failed.

Idempotency in callbacks

Inngest gives idempotency at two layers:

- Event-level (`event.id`): 24-hour dedupe window.
- Function-level (`idempotency` CEL expression): same window, evaluated against payload data.

Pattern: use `runId` as `event.id` so step replays in Inngest don't create duplicate Langfuse traces (already done in v3).

For the Fly→Worker callback, the Inngest step that POSTs the callback can fire multiple times if the step is replayed. Worker handler must be idempotent. Conditional UPDATE is cleanest:

```
UPDATE v3_runs
SET status = ?, result_json = ?, finished_at = ?
WHERE id = ? AND status NOT IN ('completed', 'failed', 'completed_with_errors');
```

If `changes()` returns 0, the callback was a duplicate — log it and return 200 (Inngest must see success or it'll retry). Never use upsert semantics for terminal-state writes.

Observability for debugging failures

Three log streams need correlation:

System	Identifier	Where to find it
Langfuse	`traceId = runId`	Langfuse Cloud, `thes1s-dev` project
Inngest	function run ID + event ID	Inngest dashboard, filter by `runId` in event data
Worker	`runId` in console.warn	`wrangler tail`
D1	`runId` in `v3_runs.id`	`wrangler d1 execute thes1s --command "SELECT ..."`

Single rule: `runId` is the global correlation key. Set when `/api/v3/pipeline/onepager/start` writes the D1 row, passed as the Inngest `event.id`, used as the Langfuse `trace_id`, echoed in every log line.

Log at minimum on each Anthropic call: `runId`, agent name, model, `stop_reason`, `usage`, latency. Log on every step transition. Log full request body on 4xx.

Timeouts everywhere

The cascade matters. Set the inner timeout shorter than the next layer out:

Layer	Timeout	Notes
Anthropic S..	`timeout` option (default 10 min)	Set explicitly per call (90s for One Pager, 300s for Pitch D..
Inngest func..	`timeouts.finish`	Currently ``'15m'`. Hard wall — past this, function fails and ..
Inngest step	implicit, follows function timeout	No separate step timeout
Fly request	machine-level (default 5min, configurable in `fly.toml`)	Raise to match Inngest's `timeouts.finish`
Worker requ..	no hard wall-clock for HTTP-triggered Workers as long as client connected; 30s grace ..	CPU time capped (5min on paid plan); offload long work to..

The `/api/v3/pipeline/onepager/start` Worker route returns 202 immediately — no long Worker request. Long-running work lives entirely on Fly + Inngest.

User-facing error UX

Three categories of message:

1. Transient ("we'll retry"): "Generation paused — retrying in 30s."
2. Terminal but recoverable ("try again"): "Generation failed. Your card was not charged. [Retry button]."
3. Terminal and not-recoverable: "Something went wrong. We've been notified. [Support link]."

Never expose raw error strings ("ZodError: Expected string at $.section_3.bullet_2"). Genericize for the user, log the full error. Always say explicitly "your card was not charged" when true — eliminates the support ticket.

Test patterns for agent robustness

- Replay traces: capture every Anthropic request/response pair in production (Langfuse already does this).
- Property-based tests: not "output equals X" but "output satisfies invariants" — e.g., `mos_buy_price <= sticker_price`.
- Schema regression suite: run 50+ historical prompts against the current schema; if any fail, your schema change is breaking.
- Production sampling: log 1-in-100 production traces with full inputs; spot-check weekly.
- Fault injection in CI: mock Anthropic to return 429, 500, malformed JSON, `pause_turn`.

The single most useful test: the failing-case fixture. When a real user run fails in production, capture the full DataPacket + prompt + response, freeze it as a fixture, and run it through CI on every PR. The known-verdict observatory companies already do this for verdict accuracy; extend the same idea to robustness.

Streaming Progress for Long Agent Runs

This section covers how to surface live progress during 5–10 minute multi-agent runs in the Thes1s stack (Cloudflare Pages SPA → CF Worker → Inngest Cloud → Fly.io TypeScript service → Anthropic SDK). The current production pattern is "poll D1 every 3s." Below: when that's enough, when it isn't, and exactly what to put on the wire.

Polling vs Push: The Tradeoff Curve

The current v1 pattern (frontend polls `/api/v3/pipeline/status/:runId` every ~3s, Worker reads a single `v3_runs` row from D1) is the cheapest and most resilient option available. It's good enough until any of three things become true:

1. You want sub-second feedback (token-by-token output, fast progress bars).
2. You want to push to multiple tabs/users for the same run (presentation mode, collab).
3. You hit cost or rate-limit problems at scale.

For Thes1s today, none of those are true. Cloudflare D1 on the Workers Paid plan includes 25 billion rows read/month. Each individual D1 database is single-threaded — if your average query takes 1ms you can run ~1,000 queries/sec [Source: https://developers.cloudflare.com/d1/reference/faq/]. 1,000 users polling every 2s = 500 reads/sec, well inside the single-DB ceiling. Polling stays good through several thousand concurrent users.

The push alternatives, in increasing complexity:

- SSE through CF Workers — Workers have *no effective cap on SSE response duration*. A Worker can hold a `text/event-stream` open for the whole 10-minute run. Worker becomes the SSE server; polls D1 internally (or subscribes to Inngest realtime) and pushes deltas to the browser. No Durable Object required if a single Worker invocation handles one client — but you give up multi-region failover and lose the connection on Worker restart. Watch the buffering pitfall: return `new Response(readableStream, { headers })` directly; helper wrappers and some frameworks buffer the whole response.
- WebSockets through CF — Requires Durable Objects on Cloudflare. DOs serialize stateful work per-key and are the right tool for bidirectional messaging or fan-out (one DO per `runId`, multiple browser tabs subscribe). Adds infra but solves multi-tab and cleanly survives reconnects.
- Inngest Realtime — Built into the v4 SDK. `step.realtime.publish()` from Fly, `useRealtime()` hook on the frontend with a token minted by your Worker. WebSocket-based with built-in reconnection (`reconnectMinMs: 250`, `reconnectMaxMs: 5000`), buffering (`bufferInterval` to batch high-frequency renders), and typed topics [Source: https://www.inngest.com/docs/features/realtime].

Recommendation: Stage 1 (One Pager, 90s) keep polling. Stage 2 (Pitch Deck, 10 min) and Stage 3 (Full Story, debate) — switch to Inngest Realtime + CF Worker SSE relay, OR keep polling but increase D1 write granularity. Either works; Inngest Realtime is the lower-effort path because it's the same provider that already runs the functions.

Available Progress Signals (Ranked by Granularity)

Signal	Granularity	Where it fires	Cost to wire
Anthropic `text_delta`	Token-level (~10ms apart)	Inside one Anthropic call	Free — already in SDK
Anthropic `input_json_delta`	Token-level on tool args	Inside one Anthropic call (tool use)	Free — SDK exposes `partial_json`
Anthropic `content_block_start`	Per content block	Per tool call / text block	Free
Anthropic `thinking_delta` (extended thinking)	Per thinking token	While model reasons	Free
Custom `step.run` callback	Per agent boundary	Wave/agent transitions	Implement publish() calls
Inngest step start/end	Coarse	Between Inngest steps	Logged automatically; wire to publish
Fly → Worker callback	Terminal only	Run completion/failure	Already implemented
Heartbeat	Liveness only	Every N seconds while idle	Cheap timer

Anthropic streams use SSE: events are `message_start`, `content_block_start`, `content_block_delta`, `content_block_stop`, `message_delta`, `message_stop`, plus periodic `ping` events. Tool-use deltas come as

`input_json_delta` with `partial_json` strings — accumulate and parse at `content_block_stop`, or use the SDK's `on('inputJson')` helper. Extended-thinking models additionally emit `thinking_delta` blocks; perfect for a "thinking..." indicator backed by real activity.

Granularity Per Stage

One Pager (1 agent, 90s). Don't over-engineer. Show: stage label, elapsed time, a heartbeat dot. Polling D1 every 2s is fine. If you want a "thinking..." chyron, swap to streaming and pipe `thinking_delta` count to the UI.

Pitch Deck (10 agents, 5 waves, ~10 min). This is where polling stops feeling alive. You want:

- Wave-level state (current wave, completed waves)
- Per-agent state inside the active wave (running / done / failed)
- Sub-progress on the slow agents (Risk Analyst doing N web searches)
- Cumulative cost and token counters

Best fit: Inngest Realtime channel keyed on `runId`, with topics `wave`, `agent`, `subprogress`, `tokens`, `done`. Publish from each Inngest step on entry/exit, and from inside the Anthropic loop for sub-progress.

Full Story (7 agents, debate). Phase-level: Bull → Bear → Rebuttal → Judge. Same machinery as Pitch Deck; just fewer events with more semantic weight per event. Consider streaming the *judge's* final synthesis token-by-token because that's the payload the user actually waits to read.

Data Shape — What the Frontend Actually Needs

A single `run_progress` JSON blob per run, written to D1 and/or pushed via Realtime. Keep it append-only-friendly so concurrent writes from a wave of agents don't clobber each other.

```
type RunProgress = {
  runId: string;
  ticker: string;
  stage: 'one-pager' | 'pitch-deck' | 'full-story';
  status: 'pending' | 'running' | 'completed' | 'failed' | 'completed_with_errors';
  startedAt: string;           // ISO
  updatedAt: string;           // bumped on every event
  heartbeatAt: string;         // bumped every ~10s even if idle
  // Coarse progress for the progress bar
  phase: { index: number; total: number; label: string };
  // Active agents
  agents: Record<string, AgentState>;
  // Cumulative stats
  tokens: { input: number; output: number; cached: number };
  costUsd: number;
  // Last 20 semantic events for the activity log
  events: ProgressEvent[];
  // Terminal payload, only set when status='completed'|'failed'
  result?: unknown;
  error?: { message: string; agentId?: string };
};

type AgentState = {
  id: string;                  // 'risk-analyst' | 'valuation-specialist' | ...
  displayName: string;
  status: 'pending' | 'running' | 'completed' | 'failed';
  startedAt?: string;
  finishedAt?: string;
  subprogress?: { current: number; total: number; label: string };
  lastMessage?: string;
  tokens?: { input: number; output: number };
};

type ProgressEvent = {
  id: string;                  // monotonic, used as SSE Last-Event-ID
  ts: string;
  kind: 'wave-start' | 'wave-end' | 'agent-start' | 'agent-end'
     | 'tool-call' | 'thinking' | 'log' | 'heartbeat' | 'error';
  agentId?: string;
  payload?: Record<string, unknown>;
};
```

The `events` array is bounded (last 20) so D1 row size stays small; the canonical event log can live in an append-only D1 table or just in Inngest Realtime topic history if you don't need replay.

Token Streaming UX Patterns

Anthropic's `messages.stream` gives token-by-token text. Three legitimate uses:

1. Typewriter on the final synthesis. For the One Pager and the Full Story judge output, stream `text_delta` events to the browser as the model writes. This is the ChatGPT-style UX users now expect.
2. Live-rendered partial JSON. For structured outputs, accumulate `input_json_delta.partial_json` strings and render half-built objects.

3. "Thinking..." indicator backed by real activity. Count `thinking_delta` events and surface "Reasoning... (12s)" instead of a spinner.

Skip token streaming for the 8 middle Pitch Deck agents. Their outputs are structured JSON consumed by the synthesis writer, not user-facing prose. Token streaming there adds wire cost for no UX benefit. Stream only the *synthesis* output the user reads.

Production Examples

- Cursor streams tool calls *with explanations beforehand* — the model says "I'll search the codebase for X" before running the tool, "which helps the user feel confident something is happening." Lesson: the agent's *narration* is the progress signal.
- Perplexity found that "users were more willing to wait for results if the product would display the intermediate progress." Lesson: show the *plan* upfront, mark steps done. Works perfectly for Pitch Deck's 5-wave structure.
- Devin has a Progress tab with Timelapse replay — slide a bar to scrub through past activity. Lesson: for a 10-minute run, "rewind" matters more than "live." Persist the event log to D1 so users can scrub after the fact.
- Vercel AI SDK standardized on SSE with a `start/delta/end` per text block, "step completion" parts to mark each LLM call boundary, and custom data parts for live status updates. UIs group events by a `stage` field. This is essentially the Pitch Deck schema above.

Inngest-Specific Patterns

Inngest Realtime is the natural fit since you're already on Inngest Cloud:

- Channels scope by `runId`/`userId`; topics categorize events.
- `step.realtime.publish()` is durable — memoized across step retries, won't double-fire.
- `publish()` is non-durable — use for high-frequency token streams where replay-on-retry is acceptable.
- Server-side subscription via `subscribe()` from `inngest/realtime` returns an SSE-encoded stream. Worker can subscribe and re-emit to browser, or mint a token via `getClientSubscriptionToken()` and let the browser subscribe directly.
- React hook `useRealtime()` handles reconnect, token refresh, message buffering.

```
// In agents-service/src/inngest/functions/pitch-deck.ts
const pitchChannel = realtime.channel({
  name: ({ runId }: { runId: string }) => `pitchdeck:${runId}`,
  topics: {
    wave:     { schema: WaveEventSchema },
    agent:    { schema: AgentEventSchema },
    log:      { schema: LogEventSchema },
    done:     { schema: DoneEventSchema },
  },
});

await step.realtime.publish(pitchChannel({ runId }).wave, {
  index: 2, total: 5, label: 'Wave 2 — Deep Analysis', startedAt: new Date().toISOStrin...
});
```

Heartbeats and the Silent-Generation Problem

Anthropic emits `ping` events during generation, but they're inside the SDK call — your Inngest function sees nothing for the 30–90s the model is generating. From the user's perspective the run looks hung. Two fixes:

1. Periodic heartbeat from inside the Anthropic loop. Set a `setInterval` while the stream is open that publishes `heartbeat` events every 5–10s with a token count. Cancel on `message_stop`.
2. Bump `heartbeatAt` on D1 from a separate timer. If the Worker's poll endpoint sees `heartbeatAt` more than 30s stale, the frontend can show "Connection healthy, model is generating..."

Heartbeats also let the frontend distinguish "backend dead" from "backend working silently."

Failure Mode: Frontend Hangs While Backend Is Fine

Polling pauses (tab backgrounded, network blip, sleep/wake) are common. The recovery contract:

- Frontend: every poll/stream message records `lastSeenEventId`. On reconnect, send `Last-Event-ID` header (SSE handles this automatically with EventSource) or include `?after=<id>` on the next poll. Backend replays missed events from the event log.
- UI states: "Live" / "Reconnecting..." / "Resumed" / "Stale" (no heartbeat in 60s, even if connection is up — model probably hung).
- Resume on tab refocus: re-fetch the full progress blob on `visibilitychange === 'visible'` to skip ahead.
- Graceful degradation: if SSE/Realtime fails, fall back to polling automatically.

The whole UI should never depend on a continuous connection — the canonical state is in D1, the stream is a delivery accelerator.

TL;DR Streaming Recommendation

1. One Pager: keep polling D1 every 2s. Add a token-streamed final-output typewriter via SSE-relay through the Worker for nicer UX. ~2 days of work.
2. Pitch Deck: switch to event-log architecture (`v3_run_events` table) + Inngest Realtime publishing on wave/agent boundaries. Frontend uses `useRealtime()` with polling fallback. ~1 week of work.
3. Full Story: same machinery as Pitch Deck, plus token-streamed judge synthesis.
4. Heartbeat from inside the Anthropic loop every 8s. Cheap, eliminates the "is it dead?" failure mode entirely.
5. Skip Durable Objects unless multi-tab presentation becomes a real product requirement.

Recommendation for Thes1s

This section maps the research findings to the three architectural questions on the table for the v3 backend before Pitch Deck and Full Story are migrated. Each recommendation is meant to be brainstorm input, not a decision — Kyle reviews and decides.

(a) Web search + structured output reconciliation

Recommended pattern: Pattern 1 (Auto-loop → forced fallback) inside `agents-service/src/lib/anthropic-client.ts`.

The wrapper change is roughly 80 lines of TypeScript. It replaces the current single forced call with a loop that runs

`tool_choice: 'auto'` for up to N turns, allows `web_search` and `emit_output` in the same toolset, and falls back to a single forced `emit_output` call only if the model didn't already emit. Failure modes are well-understood and well-mitigated; it's the pattern Anthropic Cookbook, LangGraph, the Vercel AI SDK, and Anthropic's own multi-agent research blog all converge on.

Why this over Pattern 2 (two-stage research → synthesis):

- Pattern 2 would be a pipeline change (a new "research" step in Inngest, separate Langfuse span). Kyle's stated constraint is "I don't want to change the pipeline."
- Pattern 1 is a wrapper-level change. The Inngest function structure stays exactly the same: `run-agent → validate-output → post-callback`. The agent runner (`agents-service/src/agents/one-pager.ts`) stays exactly the same except for removing the "web search disabled" comment.
- Pattern 1 preserves prompt caching across the loop turns (system + DataPacket cached once, reused on every turn).
- Pattern 1 unlocks extended thinking automatically because the loop uses `tool_choice: 'auto'` (forced tool_choice is incompatible with thinking).

Why this over Pattern 3 (native structured outputs / `output_config.format`):

- Native structured outputs is a real upgrade to consider, but it has the *same* incompatibility with `web_search` as forced tool_choice. So it doesn't solve the search-vs-output problem — it just changes the JSON-enforcement mechanism on the synthesis turn.
- It does have one strict win: cleaner code (no fake emit_output tool), real type inference via `client.messages.parse()`, decoder-level enforcement. Recommendation: do Pattern 1 first to unblock web search, then in a follow-up consider migrating the synthesis call from forced `tool_choice` to `output_config.format`. Two separate decisions.

Search caps:

- One Pager: `max_uses: 8`, `maxResearchTurns: 8`. Budget ~$0.30–$0.50 in search-related cost.
- Pitch Deck specialist: `max_uses: 3`, `maxResearchTurns: 5`. Specialists already have DataPacket; web search fills gaps.
- Full Story bull/bear: `max_uses: 5`. Judge: `max_uses: 2` (judge reasons over prior turns).

System prompt nudge: Add to each agent's system prompt (a content-only change, fully production-portable per Kyle's feedback memory): "Before emitting your structured output, you should perform at least N web searches to ground your analysis in current information. Do not emit the output tool until you have completed your research." This mitigates the "lazy emit" failure mode where the model emits on turn 1 from training data.

Estimated cost lift / quality lift:

- Re-enabling web search lifts One Pager quality from "training data only" to "current + verifiable." This is the difference between shippable and not.
- Unlocking extended thinking on Sonnet adds +5–8 quality points on analytical tasks, at +$0.15 per agent on a 10K thinking budget. Apply selectively (Risk Analyst, Valuation Specialist).
- Cost increase per One Pager: ~$0.20 (search fees + thinking). Per Pitch Deck: ~$1.65 (mostly search across 10 agents). Per Full Story: ~$1.40. All within the "sustainable" band per Vellum benchmarks.

Failure-mode plumbing:

- Detect `pause_turn` and continue the loop (this is the Anthropic server-side iteration cap).

- 4xx → `NonRetriableError` (already implemented).
- 429 → `RetryAfterError` with the Retry-After header value.
- Schema validation failures → reflect-and-retry up to 3 attempts; then `NonRetriableError`.
- Hard turn-count cap (e.g., 10 turns) + token budget cap (e.g., 200K tokens) inside the loop, with a final forced emit if either is hit.

(b) Streaming progress

Recommended approach: differentiated by stage.

For the One Pager (90s), keep polling. Add a heartbeat from inside the Anthropic loop every 8 seconds (publishes a `heartbeat` event with cumulative token count) so the frontend can distinguish "backend hung" from "model generating." Optionally add token-streamed typewriter for the final output via SSE-relay through the Worker. Total work: ~2 days.

For Pitch Deck and Full Story, adopt Inngest Realtime + an append-only event log table:

1. New D1 table `v3_run_events` — append-only, keyed on `(run_id, seq)`. Every wave start/end, agent start/end, sub-progress event, heartbeat, error. Solves the concurrency problem (10 agents in a wave updating the same row) by giving each agent its own writes. Doubles as the activity log for replay/scrub.
2. Inngest Realtime channel keyed on `runId` — Fly publishes events via `step.realtime.publish()` (durable; survives step replays). Topics: `wave`, `agent`, `subprogress`, `tokens`, `heartbeat`, `done`.
3. Worker SSE relay endpoint — `GET /api/v3/pipeline/stream/:runId` opens an SSE stream, subscribes to the Inngest Realtime channel server-side, and re-emits to the browser. Auth via session cookie (matches existing pattern).
4. Frontend hook — `useRunProgress(runId)` subscribes to SSE, falls back to polling on connection failure. Returns the `RunProgress` JSON shape defined in the streaming section.
5. Polling fallback always works — Worker still serves `/api/v3/pipeline/status/:runId` from D1. The SSE stream is a delivery accelerator; D1 is the canonical state.

Why not WebSockets / Durable Objects: Adds infrastructure for capabilities (multi-tab, bidirectional) that aren't required for the v3 user flow.

Why not just polling: A 10-minute spinner with no semantic feedback is unacceptable UX even if the backend is fine. Inngest Realtime is built into the existing stack — it's the lowest-effort path to acceptable UX.

Data contract for the frontend (matches the schema in the streaming section):

- `RunProgress` JSON blob with `phase`, `agents`, `events`, `tokens`, `costUsd`.
- `phase` is the coarse progress bar driver: `{ index, total, label }`.
- `agents` is the per-agent state map for the activity panel.
- `events` is the last 20 semantic events for the activity log; full history available via separate `/events` endpoint if needed.

Total work for Pitch Deck/Full Story: ~1 week, broken into:

- D1 schema migration (`v3_run_events` table + new columns on `v3_runs`)
- Inngest Realtime channel setup in `agents-service`
- Worker SSE relay endpoint
- Frontend `useRunProgress` hook

- Per-stage event publishing inside the agent runners

The streaming work can be done independently of the web search work. They're orthogonal changes.

(c) Robustness patterns to adopt

These are low-risk additions that pay dividends across all three stages:

1. `pause_turn` handling in the agent loop. Server tools hit a 10-iteration cap; the loop must re-send the conversation to continue. Currently not handled because the agent never loops.
2. Schema reflect-and-retry, capped at 3 attempts. When Zod fails, append the error to the conversation and retry. After 3 failures, `NonRetriableError`.
3. Conditional UPDATE on the Worker callback handler. Idempotency for `/api/v3/pipeline/callback` — only mark complete if status not already terminal.
4. `runId` as global correlation key. Already partially done. Ensure every Anthropic call logs `runId`, agent name, model, `stop_reason`, `usage`, latency. Ensure every Inngest function logs `runId`. Ensure Worker logs `runId` on every route hit.
5. Per-run token budget circuit breaker. Track total tokens across all Anthropic calls in a single Pitch Deck run; abort if it exceeds 2M (very high ceiling — anything past this is a runaway loop, not a real run).
6. Partial success handling. Add `failed_sections JSON` column to `v3_runs`. Add `completed_with_errors` status. Inngest `onFailure` writes partial state instead of just marking failed. Synthesis Writer prompt updated to handle missing sections gracefully.
7. Replay-trace test fixtures. When a real production run fails, capture the full DataPacket + prompt + response and freeze it as a CI fixture. Extends the existing `observatory/known-verdicts.json` pattern.
8. Heartbeat from inside the Anthropic loop. Eliminates the "model is generating, looks dead" failure mode without restructuring anything.

None of these touch the agent prompts, the orchestration, or the pipeline shape. All are pure infrastructure hardening.

What NOT to do

Things the research surfaced but that are worth explicitly *not* doing, given the project state:

- Don't migrate to LangGraph or another framework. The research confirms LangGraph implements Pattern 1 internally — that's the value. Hand-rolling the same pattern in `anthropic-client.ts` is ~80 lines and gives full control over Langfuse tracing, cost protection, and Inngest integration.
- Don't use MCP for in-process tools. MCP is for external tool servers. Thes1s tools are JS functions; direct in-process calls are simpler, cheaper, lower latency.
- Don't add Durable Objects to the streaming path. Durable Objects are the right tool for multi-tab, bidirectional, geographically-aware state — none of which are v3 requirements.
- Don't switch to OpenAI for the synthesis stage to dodge the structured-outputs incompatibility. The cost of leaving Anthropic is loss of prompt caching (the single biggest cost lever in the pipeline) and complexity of two-provider observability.
- Don't pre-build the "research dossier shared across 10 agents" pattern (Pattern 2 / shared retrieval). Kyle has explicitly

ruled out pipeline-level changes in this round. Pattern 1 keeps each agent autonomous as designed.

- Don't enable Tree of Thoughts, full Reflexion, or Chain of Verification. Cost ceilings without commensurate quality lift for analytical-research agents at this scale.

References

Anthropic Documentation

- [Building Effective Agents](https://www.anthropic.com/research/building-effective-agents)
- [How we built our multi-agent research system](https://www.anthropic.com/engineering/built-multi-agent-research-system)
- [Tool use overview](https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview)
- [Implement tool use](https://platform.claude.com/docs/en/agents-and-tools/tool-use/implement-tool-use)
- [Handle tool calls](https://platform.claude.com/docs/en/agents-and-tools/tool-use/handle-tool-calls)
- [Web search tool](https://platform.claude.com/docs/en/agents-and-tools/tool-use/web-search-tool)
- [Code execution tool](https://platform.claude.com/docs/en/agents-and-tools/tool-use/code-execution-tool)
- [How tool use works](https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works)
- [Structured outputs](https://platform.claude.com/docs/en/build-with-claude/structured-outputs)
- [Prompt caching](https://platform.claude.com/docs/en/build-with-claude/prompt-caching)
- [Extended thinking](https://platform.claude.com/docs/en/build-with-claude/extended-thinking)
- [Effort levels](https://platform.claude.com/docs/en/build-with-claude/effort)
- [Streaming Messages](https://platform.claude.com/docs/en/api/messages-streaming)
- [Handling stop reasons](https://platform.claude.com/docs/en/build-with-claude/handling-stop-reasons)
- [Rate limits](https://platform.claude.com/docs/en/api/rate-limits)
- [MCP connector](https://platform.claude.com/docs/en/agents-and-tools/mcp-connector)
- [Pricing](https://www.anthropic.com/pricing)
- [What's new in Claude 4.7](https://platform.claude.com/docs/en/about-claude/models/whats-new-claude-4-7)

Anthropic Cookbook & SDK

- [Anthropic Cookbook — extracting_structured_json.ipynb](https://github.com/anthropics/anthropic-cookbook/blob/main/tool_use/extracting_structured_json.ipynb)
- [Anthropic Cookbook — customer_support_agent.ipynb](https://github.com/anthropics/anthropic-cookbook/blob/main/agents/customer_support_agent.ipynb)
- [Anthropic SDK TypeScript](https://github.com/anthropics/anthropic-sdk-typescript)

Research Papers

- [ReAct (Yao et al. 2022)](https://arxiv.org/abs/2210.03629)
- [Reflexion (Shinn et al. 2023)](https://arxiv.org/abs/2303.11366)
- [Chain of Verification (Dhuliawala et al. 2023)](https://arxiv.org/abs/2309.11495)

Frameworks & Tools

- [LangChain Anthropic integration](https://python.langchain.com/docs/integrations/chat/anthropic/)
- [LangGraph Plan-and-Execute tutorial](https://langchain-ai.github.io/langgraph/tutorials/plan-and-execute/plan-and-execute/)
- [Vercel AI SDK — generating structured data](https://ai-sdk.dev/docs/ai-sdk-core/generating-structured-data)
- [Vercel AI SDK Stream Protocol](https://ai-sdk.dev/docs/ai-sdk-ui/stream-protocol)
- [Inngest Realtime](https://www.inngest.com/docs/features/realtime)
- [Inngest Errors documentation](https://www.inngest.com/docs/features/inngest-functions/error-retries/inngest-errors)
- [Inngest Idempotency guide](https://www.inngest.com/docs/guides/handling-idempotency)
- [Inngest agent tool loops](https://www.inngest.com/docs/ai-patterns/agent-tool-loops)

Infrastructure

- [Cloudflare Agents HTTP/SSE](https://developers.cloudflare.com/agents/api-reference/http-sse/)
- [Cloudflare Durable Objects WebSockets](https://developers.cloudflare.com/durable-objects/best-practices/websockets/)
- [Cloudflare D1 FAQ](https://developers.cloudflare.com/d1/reference/faq/)
- [Cloudflare D1 pricing](https://developers.cloudflare.com/d1/platform/pricing/)
- [Cloudflare Workers limits](https://developers.cloudflare.com/workers/platform/limits/)

Industry & Production Examples

- [Vellum — Structured outputs on Anthropic](https://www.vellum.ai/blog/structured-outputs-anthropic)
- [Vellum — Agent cost benchmarks 2026](https://www.vellum.ai/blog/agent-cost-benchmarks-2026)
- [Hamel Husain — Evals FAQ](https://hamel.dev/blog/posts/evals-faq/)
- [Cognition — Lessons from building Devin](https://cognition.ai/blog)
- [Cursor AI IDE architecture (Shrivu Shankar)](https://blog.sshh.io/p/how-cursor-ai-ide-works)
- [Perplexity Pro Search case study (LangChain)](https://www.langchain.com/breakoutagents/perplexity)
- [Devin product analysis (P. Paolo)](https://ppaolo.substack.com/p/in-depth-product-analysis-devin-cognition-labs)
- [OpenAI Agents Guide](https://platform.openai.com/docs/guides/agents)
- [OpenAI Structured Outputs (contrast)](https://platform.openai.com/docs/guides/structured-outputs)

Robustness & Operations

- [How to Fix Claude API 429 Rate Limit Error (2026)](https://www.aifreeapi.com/en/posts/claude-api-429-error-fix)
- [AI Agent Error Handling Patterns — Kevin Tan](https://blog.jztan.com/ai-agent-error-handling-patterns/)
- [Error Recovery and Fallback Strategies —

gocodeo](https://www.gocodeo.com/post/error-recovery-and-fallback-strategies-in-ai-agent-development)

- [Multi-Agent System Reliability — Maxim AI](https://www.getmaxim.ai/articles/multi-agent-system-reliability-failure-patterns-root-causes-and-production-validation-strategies/)
- [PraisonAI Graceful Degradation Patterns](https://docs.praison.ai/docs/best-practices/graceful-degradation)
- [Idempotent Cloud Functions for Duplicate Event Deliveries — OneUptime](https://oneuptime.com/blog/post/2026-02-17-how-to-implement-idempotent-cloud-functions-to-handle-duplicate-event-deliveries/view)

Sources Not Reachable / User Verification Needed

None of the key sources were blocked during this research run. All cited URLs were directly accessed by the research subagents.